

Université Hassan I
École Nationale des Sciences Appliquées de Berrechid

Compte rendu

Module : sql avancee

réalisé par : Sara NORELYAQINE

Année universitaire : 2025 / 2026

Table des matières

0.1	TP1 -sous requête-	2
0.1.1	exercice 1	2
0.1.2	exercice 2	3
0.2	TP2 -Les Vues en Bases de Données-	4
0.2.1	exercice 5	4
0.2.2	exercice 6	5
0.3	TP3 -Vues et index-	7
0.3.1	exercice 1	7
0.4	TP4 -Plsql	9
0.5	TP5 -cursuers-	10
0.5.1	exercice 1	10
0.5.2	exercice 2	11
0.6	TP6 -PLSQLCOURSEURS-	13
0.6.1	exercice 1	13
0.6.2	exercice 2	15
0.7	TP7 -precedure-	17
0.7.1	exercice 1	17
0.7.2	exercice 2	17
0.7.3	exercice 3	18
0.7.4	exercice 4	18
0.7.5	exercice 5	18
0.7.6	exercice 6	19
0.7.7	exercice 7	20
0.7.8	exercice 8	20
0.7.9	exercice 9	21
0.7.10	exercice 10	21
0.8	TP8 -triggers-	22

0.1 TP1 -sous requête-

0.1.1 exercice 1

```
1      -- 1. Num ro , nom et sexe des tudians dont la date de naissance
      n'est pas connue
2 SELECT E.Numetu, E.Nometu, S.Lbsexe
3 FROM ETUDIANT E
4 JOIN SEXE S ON E.Cdsexe = S.Cdsexe
5 WHERE E.Dtnaiss IS NULL;
6
7 -- 2. Nom et date de naissance des tudians dont la 2 me lettre du
      nom est 'a'
8 SELECT Nometu, Dtnaiss
9 FROM ETUDIANT
10 WHERE Nometu LIKE '_a%';
11
12 -- 3. Nom des mati res qui ont un coefficient compris entre 1 et 2
13 SELECT Nomat
14 FROM MATIERE
15 WHERE Coeff BETWEEN 1 AND 2;
16
17 -- 4. Afficher les notes de l' tudiant num ro 1 qui sont gales aux
      notes de l' tudiant num ro 2
18 SELECT Note
19 FROM NOTES
20 WHERE Numetu = 1
21 AND Note IN (SELECT Note FROM NOTES WHERE Numetu = 2);
22
23 -- 5. Afficher les notes de l' tudiant num ro 1 qui sont sup rieures
      aux notes de l' tudiant num ro 2 (au moins une)
24 SELECT Note
25 FROM NOTES
26 WHERE Numetu = 1
27 AND Note > ANY (SELECT Note FROM NOTES WHERE Numetu = 2);
28
29 -- 6. Afficher les notes de l' tudiant num ro 1 qui sont inf rieures
      toutes les notes de l' tudiant num ro 9
30 SELECT Note
31 FROM NOTES
32 WHERE Numetu = 1
33 AND Note < ALL (SELECT Note FROM NOTES WHERE Numetu = 9);
34
35 -- 7. Afficher toutes les informations sur les tudians qui n'ont
      aucune note
36 SELECT *
37 FROM ETUDIANT
38 WHERE Numetu NOT IN (SELECT DISTINCT Numetu FROM NOTES);
39
40 -- 8. Afficher quel tait l' ge moyen des gar ons et des filles au
      premier janvier 2000
41 SELECT S.Lbsexe, AVG(2000 - EXTRACT(YEAR FROM E.Dtnaiss)) AS AgeMoyen
42 FROM ETUDIANT E
43 JOIN SEXE S ON E.Cdsexe = S.Cdsexe
44 GROUP BY S.Lbsexe;
45
46 -- 9. Afficher le nom et le grade des enseignants d'histoire
47 SELECT Nomens, Grade
```

```

48 FROM ENSEIGNANT
49 WHERE Numens IN (SELECT Numens FROM MATIERE WHERE Nomat = 'Histoire');
50
51 -- 10. Afficher les noms et num ro des tudians qui n'ont pas de
52 notes en Sociologie
53 SELECT Numetu, Nometu
54 FROM ETUDIANT
55 WHERE Numetu NOT IN (
56 SELECT Numetu
57 FROM NOTES
58 WHERE Numat IN (SELECT Numat FROM MATIERE WHERE Nomat = 'Sociologie'
59 )
60 );

```

0.1.2 exercice 2

```

1 -- 11. Lieu des d partements dans lesquels des employ s touchent
2 une commission (sous-interrogation)
3 SELECT Lieu
4 FROM DEPT
5 WHERE NumDept IN (SELECT DISTINCT NumDept FROM EMP WHERE Commission IS
6 NOT NULL AND Commission > 0);
7
8 -- 12. Noms et lieux des d partements avec au moins un ing nieur
9 -- Version Sous-requ te :
10 SELECT NomDept, Lieu
11 FROM DEPT
12 WHERE NumDept IN (SELECT NumDept FROM EMP WHERE Poste = 'Ing nieur');
13
14 -- Version Jointure :
15 SELECT DISTINCT D.NomDept, D.Lieu
16 FROM DEPT D
17 JOIN EMP E ON D.NumDept = E.NumDept
18 WHERE E.Poste = 'Ing nieur';
19
20 -- 13. Noms et lieux des d partements dans lesquels il n'y a pas d'
21 ing nieur
22 SELECT NomDept, Lieu
23 FROM DEPT
24 WHERE NumDept NOT IN (SELECT DISTINCT NumDept FROM EMP WHERE Poste = '
25 Ing nieur' AND NumDept IS NOT NULL);
26
27 -- 14. Liste des employ s qui gagnent moins de 50% du salaire de leur
28 sup rieur direct (requ te synchronis e)
29 SELECT E1.NomE, E1.Salaire
30 FROM EMP E1
31 WHERE E1.Salaire < 0.5 * (
32 SELECT E2.Salaire
33 FROM EMP E2
34 WHERE E2.Matr = E1.Sup
35 );
36
37 -- 15. Noms des employ s qui gagnent plus que tous les commerciaux
38 SELECT NomE
39 FROM EMP
40 WHERE Salaire > ALL (SELECT Salaire FROM EMP WHERE Poste = 'Commercial')
41 ;

```

```

36
37 -- 16. Noms des employ s qui gagnent plus que tous les commerciaux de
    leur d partement
38 -- Solution qui INCLUT les employ s des d partements sans commerciaux
    :
39 SELECT E1.NomE
40 FROM EMP E1
41 WHERE E1.Salaire > ALL (
42     SELECT E2.Salaire
43     FROM EMP E2
44     WHERE E2.Poste = 'Commercial' AND E2.NumDept = E1.NumDept
45 );
46
47 -- Solution qui EXCLUT les employ s des d partements sans commerciaux
    :
48 SELECT E1.NomE
49 FROM EMP E1
50 WHERE E1.Salaire > ALL (
51     SELECT E2.Salaire
52     FROM EMP E2
53     WHERE E2.Poste = 'Commercial' AND E2.NumDept = E1.NumDept
54 )
55 AND EXISTS (
56     SELECT 1
57     FROM EMP E3
58     WHERE E3.Poste = 'Commercial' AND E3.NumDept = E1.NumDept
59 );
60
61 -- 17. Employ s ayant le m me poste et le m me sup rieur que BERGER
    (plusieurs colonnes)
62 SELECT NomE
63 FROM EMP
64 WHERE (Poste, Sup) = (SELECT Poste, Sup FROM EMP WHERE NomE = 'BERGER')
65 AND NomE <> 'BERGER';

```

0.2 TP2 -Les Vues en Bases de Données-

0.2.1 exercice 5

```

1 -- 1. Cr er une vue empxpublic sans le salaire
2 CREATE VIEW emp_public AS
3 SELECT emp_id, nom, dept_id
4 FROM Employe;
5
6
7 -- 2. Discuter : une vue am liore-t-elle les performances ? [
8 /*
9     R PONSE TH ORIQUE :
10     Non, une vue classique n'am liore pas nativement les performances.
11     - Une vue classique est une table virtuelle (une requ te SQL
        stock e) sans donn es physiques.
12     - chaque interrogation, le SGBD fusionne la vue avec la requ te
        globale et r ex cute
13     l'ensemble sur les tables physiques sous-jacentes (Employe,
        Departement).

```

```

14 - Le co t CPU/requ te reste identique. Elle sert l'abstraction
    et la s curit ,
15 pas l'optimisation (contrairement aux vues mat rialis es).
16 */

```

0.2.2 exercice 6

```

1  -- 1. Architecture bas e sur les vues pour les contraintes de
    s curit [cite: 114]
2
3  -- [Contrainte] Les employ s ne doivent pas voir les salaires des
    autres
4  -- On utilise la vue 'emp_public' cr e e l'exercice 5 (Masquage
    vertical de colonne).
5
6  -- [Contrainte] Les managers doivent voir uniquement les employ s de
    leur d partement
7  -- Approche par vue dynamique/synchronis e bas e sur le contexte de l'
    utilisateur connect :
8  CREATE VIEW vue_manager_dept AS
9  SELECT e.*
10 FROM Employe e
11 WHERE e.dept_id = (
12     SELECT m.dept_id
13     FROM Manager m
14     WHERE m.nom_utilisateur = USER -- 'USER' r cup re le pseudo de la
        session active
15 );
16
17 -- [Contrainte] Le service RH doit acc der toutes les informations
18 -- Les RH poss dent un acc s direct aux tables physiques ou une vue
    globale non filtr e.
19
20
21 -- 2. Limites des vues classiques dans un contexte de tr s grand volume
    de donn es
22 /*
23 - Recalcul syst matique : Les jointures et agr gats (comme MIN, MAX
    , AVG) sont recalcul s
24     chaque appel sur des centaines de milliers de lignes, ce qui
    sature le CPU.
25 - Absence d'indexation : On ne peut pas indexer directement une vue
    classique pour acc l rer
26     les recherches sp cifiques.
27 - Trafic r seau : Aucun stockage local du r sultat, for ant des
    scans de tables lourds chaque fois.
28 */
29
30
31 -- 3. Int r t des vues mat rialis es dans ce contexte
32 /*
33 - Stockage physique : Contrairement la vue classique, elle stocke
    le r sultat sur disque.
34 - Id al pour les statistiques RH : Pour la vue 'stat_dept', le
    calcul des moyennes/salaires
35     n'est fait qu'une seule fois (au moment du rafra chissement) et
    non chaque consultation.

```

```

36  - Gain de performance majeur : Les lectures deviennent instantanées
    sur les gros volumes.
37  */
38  -- Exemple de mise en œuvre pour les statistiques d'entreprise :
39  CREATE MATERIALIZED VIEW mv_stat_dept
40  REFRESH FAST ON COMMIT -- Rafraichissement optimisé lors des mises
    jour
41  AS
42  SELECT d.nom_dept, MIN(e.salaire) AS salaire_min, MAX(e.salaire) AS
    salaire_max, AVG(e.salaire) AS salaire_moyen
43  FROM Employe e
44  JOIN Departement d ON e.dept_id = d.dept_id
45  GROUP BY d.nom_dept;
46
47
48  -- 4. Comparer l'utilisation des vues avec celle des privilèges (GRANT/
    REVOKE)
49  /*
50  - Granularité : Les privilèges GRANT/REVOKE classiques agissent sur
    toute une table ou une colonne,
51  mais ne savent pas filtrer le *contenu* des lignes (ex: interdire
    les employés hors de Paris).
52  - Complémentarité : La vue permet un filtrage de contenu très fin
    (lignes + colonnes).
53  La stratégie idéale combine les deux :
54  1. Créer la vue filtrée.
55  2. Accorder le privilège sur la vue : GRANT SELECT ON
    vue_manager_dept TO role_manager;
56  3. Révoquer l'accès à la table source : REVOKE ALL ON Employe
    FROM role_manager;
57  */
58
59
60  -- 5. Une vue peut-elle garantir elle seule la sécurité complète d'
    'un système ?
61  /*
62  Non. Une vue n'est qu'un filtre logique de surface.
63  - Si les privilèges sur les tables sources sont mal configurés, un
    utilisateur peut contourner la vue.
64  - Elle ne protège pas contre les injections SQL, le vol d'
    identifiants ou les failles réseau.
65  - La sécurité doit être globale : chiffrement, pare-feu, et
    politique stricte de moindres privilèges.
66  */
67
68
69  -- 6. Stratégie d'optimisation globale proposée
70  /*
71  Pour un grand volume international de données, la meilleure
    infrastructure est:
72
73  A. Le Partitionnement : Découper physiquement la table 'Employe' par
    région ou par 'dept_id'
74  pour éviter au SGBD de scanner le monde entier lors d'une
    requête locale.
75
76  B. L'Indexation : Placer des index B-Tree sur les clés de jointures
    et de filtres fréquents :

```

```

77 CREATE INDEX idx_emp_dept ON Employe(dept_id);
78 CREATE INDEX idx_dept_ville ON Departement(ville);
79
80 C. Le Row-Level Security (RLS) / VPD : Utiliser la s e curit au
81 niveau des lignes native du SGBD
82 plut t que de multiplier les vues classiques individuelles, pour
de meilleures performances.
*/

```

0.3 TP3 -Vues et index-

0.3.1 exercice 1

```

1 -- 1. Cr ation de la vue V_EMP avec calcul des GAINS et jointure
du Lieu
2 -- Note : On utilise COALESCE ou NVL pour traiter les commissions NULL
comme tant gales 0.
3 CREATE VIEW V_EMP AS
4 SELECT
5 E.Matr,
6 E.NomE,
7 E.NumDept,
8 (E.Salaire + COALESCE(E.Commission, 0)) AS GAINS,
9 D.Lieu
10 FROM EMP E
11 JOIN DEPT D ON E.NumDept = D.NumDept;
12
13
14 -- 2. S lection des lignes de V_EMP dont le salaire total (GAINS) est >
10.000
15 SELECT * FROM V_EMP
16 WHERE GAINS > 10000;
17
18
19 -- 3. Essai de mise jour du nom de l'employ MARTIN travers la
vue V_EMP
20 UPDATE V_EMP
21 SET NomE = 'MARTIN_MODIFIE'
22 WHERE NomE = 'MARTIN';
23
24 /*
25 ANALYSE TH ORIQUE DE LA QUESTION 3 :
26 - Que se passe-t-il ? La mise jour de la colonne 'NomE' r ussit
sur la plupart des SGBD modernes (comme Oracle).
27 - Pourquoi ? Bien que la vue 'V_EMP' comporte une jointure (EMP et
DEPT), la colonne 'NomE' appartient la table
28 'EMP', qui est ici consid r e comme une "table de pr servation
de cl " (key-preserved table).
29 - Limite : En revanche, il serait strictement impossible de modifier
la colonne 'Lieu' travers cette vue,
30 car cela impacterait la table non pr serv e 'DEPT'.
31 */
32
33
34 -- 4. Cr ation de la vue VEMP10 sans l'option CHECK

```



```

35 CREATE VIEW VEMP10 AS
36 SELECT * FROM EMP
37 WHERE NumDept = 10;
38
39 -- Insertion de l'employ SOULIER dans le d partement 20      travers la
    vue
40 INSERT INTO VEMP10 (Matr, NomE, NumDept, Salaire)
41 VALUES (9999, 'SOULIER', 20, 3000);
42
43 /*
44     ANALYSE DE L'INSERTION (SANS WITH CHECK OPTION) :
45     - Recherche via la vue VEMP10 : L'employ SOULIER n'appara t PAS
        dans la vue (SELECT * FROM VEMP10).
46     Pourquoi ? La vue filtre strictement les lignes o NumDept = 10.
47     - Recherche via la table EMP : L'employ SOULIER appara t bien dans
        la table physique EMP (SELECT * FROM EMP).
48     Pourquoi ? Comme l'option de v rification manquait, le SGBD a
        accept l' criture dans la table de base,
49     m me si la ligne chappe instantan ment au filtre de la vue.
50 */
51
52
53 -- 5. Destruction et recr ation de la vue VEMP10 avec l'option CHECK
54 DROP VIEW VEMP10;
55
56 CREATE VIEW VEMP10 AS
57 SELECT * FROM EMP
58 WHERE NumDept = 10
59 WITH CHECK OPTION;
60
61
62 -- 6. Essai d'insertion de l'employ BALARD pour le d partement 30 [
    cite: 133]
63 INSERT INTO VEMP10 (Matr, NomE, NumDept, Salaire)
64 VALUES (8888, 'BALARD', 30, 3500);
65
66 /*
67     ANALYSE DE L'INSERTION (AVEC WITH CHECK OPTION) :
68     - Que se passe-t-il ? Le SGBD bloque l'insertion et renvoie une
        erreur (ex: "ORA-01402: view WITH CHECK OPTION where-clause
        violation").
69     - Pourquoi ? L'option 'WITH CHECK OPTION' interdit toute insertion ou
        modification de donn es qui ne
70     validerait pas la clause WHERE de la vue (ici, NumDept doit
        impr ativement tre gal 10).
71 */
72
73 -- Essai de modification du d partement d'un employ visualis l'
    aide de cette vue
74 UPDATE VEMP10
75 SET NumDept = 30
76 WHERE NomE = 'UN_EMPLOYE_DU_DEPT_10';
77
78 /*
79     ANALYSE DE LA MODIFICATION :
80     - Que se passe-t-il ? L'op ration choue galement avec le m me
        message d'erreur.

```

```

81  - Pourquoi ? Changer le d partement d'un employ du 10 vers le 30
82    le ferait "dispara tre" de la vue,
83    ce que le verrou 'WITH CHECK OPTION' emp che explicitement.
84  */
85
86  -- 7. Liste des salaires avec le pourcentage par rapport au total de
87    leur d partement [cite: 135]
88
89  -- SOLUTION A : En cr ant d'abord une vue interm diaire pour stocker
90    la somme des salaires par d partement
91  CREATE VIEW V_TOT_SAL_DEPT AS
92  SELECT NumDept, SUM(Salaire) AS TotalSalDept
93  FROM EMP
94  GROUP BY NumDept;
95
96  -- Requ te finale utilisant la vue interm diaire :
97  SELECT
98    E.NomE,
99    E.NumDept,
100    E.Salaire,
101    V.TotalSalDept,
102    (E.Salaire / V.TotalSalDept) * 100 AS Pourcentage
103  FROM EMP E
104  JOIN V_TOT_SAL_DEPT V ON E.NumDept = V.NumDept;
105
106  -- SOLUTION B : Solution alternative SANS vue (avec un SELECT embo t
107    dans la clause FROM)
108  SELECT
109    E.NomE,
110    E.NumDept,
111    E.Salaire,
112    T.TotalSalDept,
113    (E.Salaire / T.TotalSalDept) * 100 AS Pourcentage
114  FROM EMP E
115  JOIN (
116    SELECT NumDept, SUM(Salaire) AS TotalSalDept
117    FROM EMP
118    GROUP BY NumDept
119  ) T ON E.NumDept = T.NumDept;

```

0.4 TP4 -Plsql

```

1  SET SERVEROUTPUT ON;
2
3  DECLARE
4    -- 1. Cr ation du RECORD personnalis
5    TYPE emp_record_type IS RECORD (
6      nom          VARCHAR2(50),
7      salaire      NUMBER(10, 2)
8    );
9
10   -- 2. Cr ation de la TABLE de ce RECORD (Table associative)
11   TYPE table_emp_type IS TABLE OF emp_record_type
12   INDEX BY BINARY_INTEGER;
13

```

```

14  v_employes      table_emp_type;
15  v_somme         NUMBER := 0;
16  v_moyenne       NUMBER := 0;
17  v_salaire_max   NUMBER := 0;
18  v_nom_max       VARCHAR2(50);
19 BEGIN
20  -- 3. Insérer 5 employés manuellement dans la table (en mémoire)
21  v_employes(1).nom := 'Alice';    v_employes(1).salaire := 3500;
22  v_employes(2).nom := 'Bob';      v_employes(2).salaire := 4200;
23  v_employes(3).nom := 'Charlie';  v_employes(3).salaire := 2800;
24  v_employes(4).nom := 'Diane';    v_employes(4).salaire := 5100;
25  v_employes(5).nom := 'Eric';     v_employes(5).salaire := 3900;
26
27  -- Parcours de la table pour les calculs
28  FOR i IN 1..v_employes.COUNT LOOP
29      -- Cumul pour le calcul de la moyenne
30      v_somme := v_somme + v_employes(i).salaire;
31
32      -- 5. Identifier le salaire maximum
33      IF v_employes(i).salaire > v_salaire_max THEN
34          v_salaire_max := v_employes(i).salaire;
35          v_nom_max      := v_employes(i).nom;
36      END IF;
37  END LOOP;
38
39  -- 4. Calculer le salaire moyen
40  v_moyenne := v_somme / v_employes.COUNT;
41
42  -- Affichage des résultats
43  DBMS_OUTPUT.PUT_LINE('--- RÉSULTATS DE SYNTHÈSE ---');
44  DBMS_OUTPUT.PUT_LINE('Nombre d''employés : ' || v_employes.COUNT);
45  DBMS_OUTPUT.PUT_LINE('Salaire moyen      : ' || ROUND(v_moyenne, 2)
46  || ' ');
47  DBMS_OUTPUT.PUT_LINE('Salaire maximum   : ' || v_salaire_max || ' '
48  || '(Détenu par ' || v_nom_max || ')');
49 END;
50 /

```

0.5 TP5 -cursuers-

0.5.1 exercice 1

```

1  -- 1. Liste de toutes les commandes
2  DECLARE
3      CURSOR c_commandes IS
4          SELECT Numcmd, Datcmd, Numfour, Mncmd
5          FROM Commande;
6      v_cmd c_commandes%ROWTYPE;
7  BEGIN
8      OPEN c_commandes;
9      LOOP
10         FETCH c_commandes INTO v_cmd;
11         EXIT WHEN c_commandes%NOTFOUND;
12         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_cmd.Numcmd || ' -
            Date: ' || v_cmd.Datcmd || ' - Fournisseur: ' || v_cmd.
            Numfour || ' - Montant: ' || v_cmd.Mncmd);

```

```

13     END LOOP;
14     CLOSE c_commandes;
15 END;
16 /
17
18 -- 2. Liste des commandes ayant un montant sup rieur      la moyenne
19 DECLARE
20     CURSOR c_cmd_sup_moy IS
21         SELECT Numcmd, Mncmd
22         FROM Commande
23         WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
24     v_numcmd Commande.Numcmd%TYPE;
25     v_mncmd Commande.Mncmd%TYPE;
26 BEGIN
27     OPEN c_cmd_sup_moy;
28     LOOP
29         FETCH c_cmd_sup_moy INTO v_numcmd, v_mncmd;
30         EXIT WHEN c_cmd_sup_moy%NOTFOUND;
31         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_numcmd || ' - Montant
                                : ' || v_mncmd);
32     END LOOP;
33     CLOSE c_cmd_sup_moy;
34 END;
35 /
36
37 -- 3. Liste des commandes ayant un fournisseur parisien
38 DECLARE
39     CURSOR c_cmd_paris IS
40         SELECT C.Numcmd, C.Mncmd, F.Adrfour
41         FROM Commande C
42         JOIN Fournisseurs F ON C.Numfour = F.Numfour
43         WHERE LOWER(F.Adrfour) = 'paris';
44     v_numcmd Commande.Numcmd%TYPE;
45     v_mncmd Commande.Mncmd%TYPE;
46     v_adr Fournisseurs.Adrfour%TYPE;
47 BEGIN
48     OPEN c_cmd_paris;
49     LOOP
50         FETCH c_cmd_paris INTO v_numcmd, v_mncmd, v_adr;
51         EXIT WHEN c_cmd_paris%NOTFOUND;
52         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_numcmd || ' - Montant
                                : ' || v_mncmd || ' - Adresse: ' || v_adr);
53     END LOOP;
54     CLOSE c_cmd_paris;
55 END;
56 /

```

0.5.2 exercice 2

```

1     -- 1. Liste des employ s ayant une commission (non NULL) class e
        par commission d croissante
2 DECLARE
3     CURSOR c_comm_dec IS
4         SELECT NomE, Comm
5         FROM Emp
6         WHERE Comm IS NOT NULL AND Comm > 0

```

```

7      ORDER BY Comm DESC;
8 BEGIN
9     FOR r_emp IN c_comm_dec LOOP
10        DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE || ' -
11           Commission: ' || r_emp.Comm);
12    END LOOP;
13 END;
14 /
15 -- 2. Noms des personnes embauch es depuis le 01-09-2006
16 DECLARE
17     CURSOR c_embauche IS
18         SELECT NomE, Embauche
19         FROM Emp
20         WHERE Embauche >= TO_DATE('01-09-2006', 'DD-MM-YYYY');
21 BEGIN
22     FOR r_emp IN c_embauche LOOP
23        DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE || ' - Date
24           Embauche: ' || TO_CHAR(r_emp.Embauche, 'DD-MM-YYYY'));
25    END LOOP;
26 END;
27 /
28 -- 3. Liste des employ s travaillant      Cr teil
29 DECLARE
30     CURSOR c_creteil IS
31         SELECT E.NomE, D.Lieu
32         FROM Emp E
33         JOIN Dept D ON E.NumD = D.NumD
34         WHERE LOWER(D.Lieu) LIKE 'cr%' || 'e' || 'il%'; -- S'adapte aux
35           variantes d'orthographe comme Cr eil ou Cr teil
36 BEGIN
37     FOR r_emp IN c_creteil LOOP
38        DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE || ' - Lieu: '
39           || r_emp.Lieu);
40    END LOOP;
41 END;
42 /
43 -- 4. Liste des subordonn s de "Guimezanes"
44 DECLARE
45     CURSOR c_subordonnes IS
46         SELECT NomE
47         FROM Emp
48         WHERE NumS = (SELECT NumE FROM Emp WHERE NomE = 'Guimezanes');
49 BEGIN
50     FOR r_emp IN c_subordonnes LOOP
51        DBMS_OUTPUT.PUT_LINE('Subordonn de Guimezanes: ' || r_emp.NomE
52           );
53    END LOOP;
54 END;
55 /
56 -- 5. Moyenne des salaires
57 DECLARE
58     CURSOR c_salaires IS
59         SELECT Salaire
60         FROM Emp;

```

```

60     v_total NUMBER := 0;
61     v_count NUMBER := 0;
62     v_moyenne NUMBER := 0;
63 BEGIN
64     FOR r_emp IN c_salaires LOOP
65         v_total := v_total + r_emp.Salaire;
66         v_count := v_count + 1;
67     END LOOP;
68     IF v_count > 0 THEN
69         v_moyenne := v_total / v_count;
70     END IF;
71     DBMS_OUTPUT.PUT_LINE('Moyenne des salaires de l''entreprise: ' ||
72         ROUND(v_moyenne, 2));
73 END;
74 /
75 -- 6. Nombre de commissions non NULL
76 DECLARE
77     CURSOR c_comm_count IS
78         SELECT Comm
79         FROM Emp
80         WHERE Comm IS NOT NULL;
81     v_nb_comm NUMBER := 0;
82 BEGIN
83     FOR r_emp IN c_comm_count LOOP
84         v_nb_comm := v_nb_comm + 1;
85     END LOOP;
86     DBMS_OUTPUT.PUT_LINE('Nombre total de commissions non NULL: ' ||
87         v_nb_comm);
88 END;
89 /
90 -- 7. Liste des employ s gagnant plus que la moyenne des salaires de l'
91     entreprise
92 DECLARE
93     CURSOR c_sup_moy IS
94         SELECT NomE, Salaire
95         FROM Emp
96         WHERE Salaire > (SELECT AVG(Salaire) FROM Emp);
97 BEGIN
98     FOR r_emp IN c_sup_moy LOOP
99         DBMS_OUTPUT.PUT_LINE('Employ : ' || r_emp.NomE || ' - Salaire:
100         ' || r_emp.Salaire);
101     END LOOP;
102 END;
103 /

```

0.6 TP6 -PLSQLCURSEURS-

0.6.1 exercice 1

```

1     -- cree un programme PL/SQL pour r organiser le nombre de deux
2     variables (small_nbr et big_nbr)
3 DECLARE
4     small_nbr NUMBER := 25;

```

```

4      big_nbr    NUMBER := 5;
5      temp_nbr   NUMBER;
6 BEGIN
7      IF small_nbr > big_nbr THEN
8          temp_nbr := small_nbr;
9          small_nbr := big_nbr;
10         big_nbr   := temp_nbr;
11     END IF;
12     DBMS_OUTPUT.PUT_LINE('small_nbr = ' || small_nbr);
13     DBMS_OUTPUT.PUT_LINE('big_nbr = ' || big_nbr);
14 END;
15 /
16
17 --   crire   un programme PL/SQL pour v rifier si un nombre est pair ou
        impair
18 DECLARE
19     nbr NUMBER := 3;
20 BEGIN
21     IF MOD(nbr, 2) = 0 THEN
22         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est un nombre pair
        .');
23     ELSE
24         DBMS_OUTPUT.PUT_LINE('Le nombre ' || nbr || ' est un nombre
        impair. ');
25     END IF;
26 END;
27 /
28
29 -- Ecrire un programme PL/SQL pour v rifier si une date tombe le week-
        end (SAMEDI ou DIMANCHE)
30 DECLARE
31     date1 DATE := TO_DATE('2-Mar-2024', 'DD-Mon-YYYY', '
        NLS_DATE_LANGUAGE=ENGLISH');
32     jour_semaine VARCHAR2(20);
33 BEGIN
34     jour_semaine := TO_CHAR(date1, 'DAY', 'NLS_DATE_LANGUAGE=ENGLISH');
35
36     IF TRIM(jour_semaine) IN ('SATURDAY', 'SUNDAY') THEN
37         DBMS_OUTPUT.PUT_LINE('Le jour de la date donn e est ' || TRIM(
        jour_semaine) || ': tombe le week-end. ');
38     ELSE
39         DBMS_OUTPUT.PUT_LINE('Le jour de la date donn e est ' || TRIM(
        jour_semaine) || ': ne tombe pas le week-end. ');
40     END IF;
41 END;
42 /
43
44 --   crire   un programme en PL/SQL pour copier les enregistrements de la
        table clients   une autre (tmp_clients)
45 -- Note : On suppose que la table tmp_clients existe d j   et poss de
        la m me structure.
46 DECLARE
47     CURSOR c_clients IS
48         SELECT client_id, nom, ville, age FROM clients;
49 BEGIN
50     FOR r_client IN c_clients LOOP
51         INSERT INTO tmp_clients (client_id, nom, ville, age)

```

```

52         VALUES (r_client.client_id, r_client.nom, r_client.ville,
53                 r_client.age);
54     END LOOP;
55     COMMIT;
56 END;
/

```

0.6.2 exercice 2

```

1  -- cree un programme PL/SQL pour afficher les noms de tous les
2  pays
3  DECLARE
4      CURSOR c_pays IS
5          SELECT nom FROM pays;
6  BEGIN
7      FOR r_pays IN c_pays LOOP
8          DBMS_OUTPUT.PUT_LINE('Nom du pays : ' || r_pays.nom);
9      END LOOP;
10 END;
11 /
12 -- cree un programme PL/SQL pour afficher les identifiants des pays
13 et les noms avec titre appropri
14 DECLARE
15     CURSOR c_pays_titre IS
16         SELECT pays_id, nom FROM pays;
17 BEGIN
18     DBMS_OUTPUT.PUT_LINE('=== LISTE DES PAYS ENREGISTR S ===');
19     FOR r_pays IN c_pays_titre LOOP
20         DBMS_OUTPUT.PUT_LINE('ID : ' || r_pays.pays_id || ' | Nom : ' ||
21                               r_pays.nom);
22     END LOOP;
23 END;
24 /
25 -- cree un programme en PL/SQL pour trouver le nombre de lignes
26 affect es par l utilisation de l attribut SQL%ROWCOUNT d un
27 curseur implicite
28 DECLARE
29     v_lignes_affectees NUMBER;
30 BEGIN
31     UPDATE pays
32     SET nom = nom
33     WHERE pays_id > 1000;
34
35     v_lignes_affectees := SQL%ROWCOUNT;
36     DBMS_OUTPUT.PUT_LINE('Nombre de lignes affect es par la mise
37 jour : ' || v_lignes_affectees);
38 END;
39 /
40 -- cree un programme PL/SQL pour afficher les identifiants, noms,
41 pr noms et les noms de d partements de tous les employ s
42 DECLARE
43     CURSOR c_emp_dept IS
44         SELECT e.id, e.nom AS emp_nom, e.prenom, d.nom AS dept_nom

```



```

41      FROM employees e
42      JOIN departements d ON e.departement_id = d.dep_id;
43 BEGIN
44     FOR r_data IN c_emp_dept LOOP
45         DBMS_OUTPUT.PUT_LINE('ID: ' || r_data.id || ' | Nom: ' || r_data
                                .emp_nom || ' | Pr nom: ' || r_data.prenom || ' |
                                D partement: ' || r_data.dept_nom);
46     END LOOP;
47 END;
48 /
49
50 --  creire  un programme en PL/SQL pour montrer  l utilisation  de SQL%
      FOUND afin de d terminer si  l instruction  DELETE a affect  des
      lignes
51 BEGIN
52     DELETE FROM pays
53     WHERE pays_id = 1007;
54
55     IF SQL%FOUND THEN
56         DBMS_OUTPUT.PUT_LINE('Suppression r ussie : Des lignes ont
                                t  affect es par l''instruction DELETE.');

```

```

87         UPDATE employees
88         SET salaire = salaire + 100
89         WHERE CURRENT OF c_emp_dept8;
90     ELSE
91         UPDATE employees
92         SET salaire = salaire + 50
93         WHERE CURRENT OF c_emp_dept8;
94     END IF;
95 END LOOP;
96 COMMIT;
97 END;
98 /

```

0.7 TP7 -precedure-

0.7.1 exercice 1

```

1  -- 1. Programme (bloc anonyme) pour afficher Hello
2 BEGIN
3     DBMS_OUTPUT.PUT_LINE('Hello');
4 END;
5 /
6
7 -- 2. Fonction qui retourne le carré d'un nombre
8 CREATE OR REPLACE FUNCTION carre_nombre(p_nbr IN NUMBER)
9 RETURN NUMBER IS
10 BEGIN
11     RETURN p_nbr * p_nbr;
12 END;
13 /

```

0.7.2 exercice 2

```

1  -- Proc dure avec param tre OUT
2 CREATE OR REPLACE PROCEDURE max_deux_nombres(
3     p_num1 IN NUMBER,
4     p_num2 IN NUMBER,
5     p_max OUT NUMBER
6 ) IS
7 BEGIN
8     IF p_num1 > p_num2 THEN
9         p_max := p_num1;
10    ELSE
11        p_max := p_num2;
12    END IF;
13 END;
14 /
15
16 -- Version fonction du calcul du maximum
17 CREATE OR REPLACE FUNCTION max_deux_nombres_fn(
18     p_num1 IN NUMBER,
19     p_num2 IN NUMBER
20 ) RETURN NUMBER IS
21 BEGIN

```

```

22     IF p_num1 > p_num2 THEN
23         RETURN p_num1;
24     ELSE
25         RETURN p_num2;
26     END IF;
27 END;
28 /

```

0.7.3 exercice 3

```

1  -- Proc dure qui permute deux nombres (param tres IN OUT)
2 CREATE OR REPLACE PROCEDURE permuter_nombres(
3     p_num1 IN OUT NUMBER,
4     p_num2 IN OUT NUMBER
5 ) IS
6     v_temp NUMBER;
7 BEGIN
8     v_temp := p_num1;
9     p_num1 := p_num2;
10    p_num2 := v_temp;
11 END;
12 /

```

0.7.4 exercice 4

```

1  -- Proc dure par nom d'employ (prend la premi re ligne trouv e
   en cas de doublon)
2 CREATE OR REPLACE PROCEDURE augmenter_salaire(
3     p_nom IN VARCHAR2,
4     p_taux IN NUMBER
5 ) IS
6 BEGIN
7     UPDATE EMPLOYES
8     SET SALARY = SALARY * (1 + p_taux / 100)
9     WHERE ENAME = p_nom
10    AND ROWNUM = 1;
11 END;
12 /
13
14 -- Variante utilisant l'ENUM (Cl primaire)
15 CREATE OR REPLACE PROCEDURE augmenter_salaire_by_enum(
16     p_enum IN VARCHAR2,
17     p_taux IN NUMBER
18 ) IS
19 BEGIN
20     UPDATE EMPLOYES
21     SET SALARY = SALARY * (1 + p_taux / 100)
22     WHERE ENUM = p_enum;
23 END;
24 /

```

0.7.5 exercice 5

```

1      -- Fonction de conversion MAD en EUR (1 EUR = 11 MAD) avec arrondi
      2 d ciales
2 CREATE OR REPLACE FUNCTION convertir_mad_en_eur(
3     p_montant_mad IN NUMBER
4 ) RETURN NUMBER IS
5 BEGIN
6     RETURN ROUND(p_montant_mad / 11, 2);
7 END;
8 /
9
10 -- Surcharge de la fonction pour accepter un taux variable
11 CREATE OR REPLACE FUNCTION convertir_mad_en_eur(
12     p_montant_mad IN NUMBER,
13     p_taux          IN NUMBER
14 ) RETURN NUMBER IS
15 BEGIN
16     IF p_taux > 0 THEN
17         RETURN ROUND(p_montant_mad / p_taux, 2);
18     ELSE
19         RETURN 0;
20     END IF;
21 END;
22 /

```

0.7.6 exercice 6

```

1      -- Proc dure avec curseur implicite et param tre de tri ('ASC' ou
      'DESC')
2 CREATE OR REPLACE PROCEDURE afficher_salaires_sup(
3     p_seuil IN NUMBER,
4     p_tri   IN VARCHAR2 DEFAULT 'ASC'
5 ) IS
6 BEGIN
7     IF p_seuil < 0 THEN
8         DBMS_OUTPUT.PUT_LINE('Erreur : Le montant ne peut pas tre
          n gatif. ');
9         RETURN;
10    END IF;
11
12    IF UPPER(p_tri) = 'DESC' THEN
13        FOR rec IN (SELECT * FROM EMPLOYEES WHERE SALARY > p_seuil ORDER
          BY SALARY DESC) LOOP
14            DBMS_OUTPUT.PUT_LINE('Employ : ' || rec.ENAME || ' |
          Salaire: ' || rec.SALARY);
15        END LOOP;
16    ELSE
17        FOR rec IN (SELECT * FROM EMPLOYEES WHERE SALARY > p_seuil ORDER
          BY SALARY ASC) LOOP
18            DBMS_OUTPUT.PUT_LINE('Employ : ' || rec.ENAME || ' |
          Salaire: ' || rec.SALARY);
19        END LOOP;
20    END IF;
21 END;
22 /

```

0.7.7 exercice 7

```
1      -- Fonction retournant le salaire moyen ou NULL si aucun employ
2 CREATE OR REPLACE FUNCTION salaire_moyen_dept(
3     p_dept IN VARCHAR2
4 ) RETURN NUMBER IS
5     v_moyen NUMBER;
6 BEGIN
7     SELECT AVG(SALARY)
8     INTO v_moyen
9     FROM EMPLOYES
10    WHERE DEPT = p_dept;
11
12    RETURN v_moyen;
13 EXCEPTION
14    WHEN NO_DATA_FOUND THEN
15        RETURN NULL;
16 END;
17 /
```

0.7.8 exercice 8

```
1      -- Proc dure de mise      jour de l'adresse par nom avec
2      v rification de non-vacuit
3 CREATE OR REPLACE PROCEDURE modifier_adresse_par_nom(
4     p_nom      IN VARCHAR2,
5     p_adresse  IN VARCHAR2
6 ) IS
7 BEGIN
8     IF p_adresse IS NULL OR TRIM(p_adresse) = '' THEN
9         DBMS_OUTPUT.PUT_LINE('Erreur : L''adresse fournie ne peut pas
10         tre vide.');
```

```
11     RETURN;
12 END IF;
13
14 UPDATE EMPLOYES
15 SET DRESS = p_adresse
16 WHERE ENAME = p_nom
17 AND ROWNUM = 1;
18 END;
19 /
20
21 -- M me proc dure utilisant le param tre ENUM
22 CREATE OR REPLACE PROCEDURE modifier_adresse_par_enum(
23     p_enum      IN VARCHAR2,
24     p_adresse  IN VARCHAR2
25 ) IS
26 BEGIN
27     IF p_adresse IS NULL OR TRIM(p_adresse) = '' THEN
28         DBMS_OUTPUT.PUT_LINE('Erreur : L''adresse fournie ne peut pas
29         tre vide.');
```

```
30     RETURN;
31 END IF;
32
33 UPDATE EMPLOYES
34 SET DRESS = p_adresse
```

```

32     WHERE ENUM = p_enum;
33 END;
34 /

```

0.7.9 exercice 9

```

1  -- Fonction simple retournant le nombre d'employés
2  CREATE OR REPLACE FUNCTION nb_employes_dept(
3      p_dept IN VARCHAR2
4  ) RETURN NUMBER IS
5      v_total NUMBER;
6  BEGIN
7      SELECT COUNT(*)
8      INTO v_total
9      FROM EMPLOYES
10     WHERE DEPT = p_dept;
11
12     RETURN v_total;
13 END;
14 /
15
16 -- Fonction modifiée retournant aussi le salaire moyen via une
17   proc dure ou type record
18 CREATE OR REPLACE PROCEDURE statistiques_dept(
19     p_dept          IN VARCHAR2,
20     p_nb_emp        OUT NUMBER,
21     p_sal_moyen     OUT NUMBER
22 ) IS
23 BEGIN
24     SELECT COUNT(*), NVL(AVG(SALARY), 0)
25     INTO p_nb_emp, p_sal_moyen
26     FROM EMPLOYES
27     WHERE DEPT = p_dept;
28 END;
29 /
30 -- Proc dure qui affiche pour chaque département distinct le nombre d'
31   employés
32 CREATE OR REPLACE PROCEDURE afficher_repartition_dept IS
33 BEGIN
34     FOR rec IN (SELECT DEPT, COUNT(*) AS nb FROM EMPLOYES GROUP BY DEPT)
35     LOOP
36         DBMS_OUTPUT.PUT_LINE('Département: ' || rec.DEPT || ' | Nombre
37         d''employés: ' || rec.nb);
38     END LOOP;
39 END;
40 /

```

0.7.10 exercice 10

```

1  -- Proc dure stockée avec clause SAVEPOINT
2  CREATE OR REPLACE PROCEDURE supprimer_petits_salaires(
3      p_seuil IN NUMBER
4  ) IS
5  BEGIN

```

```

6      SAVEPOINT sp_avant_suppression;
7
8      FOR rec IN (SELECT ENAME FROM EMPLOYES WHERE SALARY < p_seuil) LOOP
9          DBMS_OUTPUT.PUT_LINE('Employe supprimé : ' || rec.ENAME);
10     END LOOP;
11
12     DELETE FROM EMPLOYES
13     WHERE SALARY < p_seuil;
14
15 EXCEPTION
16     WHEN OTHERS THEN
17         ROLLBACK TO sp_avant_suppression;
18         DBMS_OUTPUT.PUT_LINE('Une erreur est survenue. Opération
19                               annulée.');
```

0.8 TP8 -triggers-

```

1      -- R g l e 1 : Interdiction de modifier note_min dans prerequis
2      CREATE OR REPLACE TRIGGER trg_regle1_note_min
3      BEFORE UPDATE OF note_min ON prerequis
4      FOR EACH ROW
5      BEGIN
6          RAISE_APPLICATION_ERROR(-20001, 'Interdiction stricte de modifier la
7          note minimale requise pour un prerequis.');
```

```

8      END;
9      /
```

```

10     -- R g l e 2 : Contrôle de effectMax lors d'une inscription
11     CREATE OR REPLACE TRIGGER trg_regle2_effectmax
12     BEFORE INSERT ON inscription
13     FOR EACH ROW
14     DECLARE
15         v_effect_max NUMBER;
16         v_actuel      NUMBER;
17     BEGIN
18         SELECT NVL(effectMax, 0) INTO v_effect_max
19         FROM module
20         WHERE code_module = :NEW.code_module;
21
22         SELECT COUNT(*) INTO v_actuel
23         FROM inscription
24         WHERE code_module = :NEW.code_module;
25
26         IF v_actuel >= v_effect_max THEN
27             RAISE_APPLICATION_ERROR(-20002, 'Impossible d''inscrire l''
```

```

28             tudiant : l''effectif maximal (effectMax) de ce module est
29             atteint.');
```

```

30         END IF;
31     END;
32     /
```

```

33     -- R g l e 3 : Création d'examen uniquement si le module possède des
34     inscrits
35     CREATE OR REPLACE TRIGGER trg_regle3_examen_inscrits
```

```

34 BEFORE INSERT ON examen
35 FOR EACH ROW
36 DECLARE
37     v_nb_inscrits NUMBER;
38 BEGIN
39     SELECT COUNT(*) INTO v_nb_inscrits
40     FROM inscription
41     WHERE code_module = :NEW.module;
42
43     IF v_nb_inscrits = 0 THEN
44         RAISE_APPLICATION_ERROR(-20003, 'Planification annul e :
            impossible de cr er un examen pour un module sans aucun
            inscrit.');
```

```

45     END IF;
46 END;
47 /
48
49 -- R gle 4 : Date d'examen post rieuse l'inscription de TOUS les
            tudiants inscrits au module
50 CREATE OR REPLACE TRIGGER trg_regle4_date_examen
51 BEFORE INSERT OR UPDATE OF date_examen ON examen
52 FOR EACH ROW
53 DECLARE
54     v_max_date_insc DATE;
55 BEGIN
56     SELECT MAX(date_inscription) INTO v_max_date_insc
57     FROM inscription
58     WHERE code_module = :NEW.module;
59
60     IF v_max_date_insc IS NOT NULL AND :NEW.date_examen <=
        v_max_date_insc THEN
61         RAISE_APPLICATION_ERROR(-20004, 'Conflit de calendrier : la date
            de l''examen doit tre strictement post rieuse toutes
            les dates d''inscription du module.');
```

```

62     END IF;
63 END;
64 /
65
66 -- R gle 5 : D tection de circuit de d pendance cyclique dans la
            table prerequis
67 CREATE OR REPLACE TRIGGER trg_regle5_circuit_prerequis
68 BEFORE INSERT OR UPDATE OF module, prerequis ON prerequis
69 FOR EACH ROW
70 DECLARE
71     v_nb_chemins NUMBER;
72 BEGIN
73     IF :NEW.module = :NEW.prerequis THEN
74         RAISE_APPLICATION_ERROR(-20005, 'Erreur de d pendance : un
            module ne peut pas tre son propre pr requis (boucle
            directe).');
```

```

75     END IF;
76
77     SELECT COUNT(*) INTO v_nb_chemins
78     FROM prerequis
79     START WITH module = :NEW.prerequis
80     CONNECT BY PRIOR prerequis = module
81     AND module = :NEW.module;
82
```



```

83     IF v_nb_chemins > 0 THEN
84         RAISE_APPLICATION_ERROR(-20005, 'D pendance cyclique d tect e
            : le pr requis choisi cr e un circuit logique interdit.');
```

85 END IF;

86 END;

87 /

88

89 -- R gle 6 : V rification des pr requis de l' tudiant lors d'une
inscription

90 CREATE OR REPLACE TRIGGER trg_regle6_verif_prerequis

91 BEFORE INSERT ON inscription

92 FOR EACH ROW

93 DECLARE

94 v_non_valide NUMBER := 0;

95 BEGIN

96 FOR r_prerequis IN (
97 SELECT p.prerequis, p.note_min
98 FROM prerequis p
99 WHERE p.module = :NEW.code_module
100) LOOP

101 DECLARE

102 v_note_obtenue NUMBER := -1;

103 BEGIN

104 SELECT MAX(ex.note) INTO v_note_obtenue
105 FROM passer ex
106 WHERE ex.id_etudiant = :NEW.id_etudiant
107 AND ex.module = r_prerequis.prerequis;

108

109 IF v_note_obtenue IS NULL OR v_note_obtenue < r_prerequis.
note_min THEN

110 v_non_valide := v_non_valide + 1;

111 END IF;

112 END;

113 END LOOP;

114

115 IF v_non_valide > 0 THEN

116 RAISE_APPLICATION_ERROR(-20006, 'Inscription refus e : l''
tudiant ne valide pas les pr requis obligatoires pour ce
module.');

117 END IF;

118 END;

119 /

120

121 -- R gle 7 : Contr le de la note minimale par rapport au pr requis
requis de la table module

122 CREATE OR REPLACE TRIGGER trg_regle7_note_min_coherence

123 BEFORE INSERT OR UPDATE OF note_min ON prerequis

124 FOR EACH ROW

125 DECLARE

126 v_base_note_min NUMBER;

127 BEGIN

128 SELECT NVL(note_min, 0) INTO v_base_note_min
129 FROM module
130 WHERE code_module = :NEW.prerequis;

131

132 IF :NEW.note_min < v_base_note_min THEN

133 RAISE_APPLICATION_ERROR(-20007, 'Incoh rence : la note minimale
exig e dans le pr requis ne peut pas tre inf rieur

```

134         la note minimale intrins que du module. ');
135     END IF;
136 END;
137 /
138 -- R g l e 8 : L' tudiant ne peut pas passer un examen s'il n'est pas
139 pr alablement inscrit
139 CREATE OR REPLACE TRIGGER trg_regle8_passer_inscription
140 BEFORE INSERT ON passer
141 FOR EACH ROW
142 DECLARE
143     v_est_inscrit NUMBER;
144 BEGIN
145     SELECT COUNT(*) INTO v_est_inscrit
146     FROM inscription
147     WHERE id_etudiant = :NEW.id_etudiant
148     AND code_module = :NEW.module;
149
150     IF v_est_inscrit = 0 THEN
151         RAISE_APPLICATION_ERROR(-20008, 'Acc s refus : l'' tudiant
152         doit obligatoirement tre inscrit au module pour composer
153         cet examen. ');
154     END IF;
155 END;
156 /
157 -- R g l e 9 : Recalcul et mise jour automatique de la moyenne de l'
158 tudiant
157 CREATE OR REPLACE TRIGGER trg_regle9_maj_moyenne
158 AFTER INSERT OR UPDATE OF note OR DELETE ON passer
159 FOR EACH ROW
160 DECLARE
161     PRAGMA AUTONOMOUS_TRANSACTION;
162     v_nouvelle_moyenne NUMBER;
163     v_id_etudiant NUMBER;
164 BEGIN
165     IF DELETING THEN
166         v_id_etudiant := :OLD.id_etudiant;
167     ELSE
168         v_id_etudiant := :NEW.id_etudiant;
169     END IF;
170
171     SELECT AVG(note) INTO v_nouvelle_moyenne
172     FROM passer
173     WHERE id_etudiant = v_id_etudiant;
174
175     UPDATE etudiant
176     SET moyenne = NVL(v_nouvelle_moyenne, 0)
177     WHERE id_etudiant = v_id_etudiant;
178
179     COMMIT;
180 END;
181 /

```